

CSE599O MemOpt

Andy Ruan

University of Washington
Seattle, Washington, USA
yhruan22@cs.washington.edu

Puyuan Yao

University of Washington
Seattle, Washington, USA
puyuan@cs.washington.edu

Abstract

Increasing the size of a language model can achieve better accuracy, but the training of larger LLMs is often bottlenecked by GPU memory. Thus, when the number of available GPUs is fixed, non-sharding techniques are needed to reduce peak GPU memory usage. While such techniques, including activation checkpointing, mixed precision training, and CPU offloading, have been proposed, given a goal of maximizing throughput while staying within the memory limit, it is non-trivial to select which subset of techniques to use or to find a good configuration for each technique.

We present MemOpt, a system that trains our homework Transformer model with activation checkpointing, mixed precision training, and/or CPU offloading. We have implemented these techniques from scratch¹ as wrapper APIs that allow easy composition. To answer the challenging question of choosing which techniques to use, we evaluate and compare the training efficiency and peak GPU memory usage of different technique combinations and configurations. Our results show: 1) mixed precision training alone has modest peak memory savings and reduces training time; 2) activation checkpointing saves slightly more memory but incurs modest slowdown; 3) CPU offloading saves the most memory but incurs significant slowdown. Thus we suggest a simple technique-choosing heuristic for navigating memory-performance tradeoff: apply mixed precision training first, add activation checkpointing if needed, and lastly also enable CPU offloading. With all techniques enabled, we are able to train a 2.8B model, which is 2.31x the size of the largest trainable model when no techniques are applied.

1 Introduction

Scaling LLMs in terms of number of parameters brings better model performance. However, as the model size increases, model weights, optimizer states, gradients, and activations all increase to take up GPU memory, which often becomes a bottlenecked in training. While sharding a model across multiple GPUs is a great way to reduce per-GPU memory usage, in settings where the number of available GPUs is limited, such as a university’s research cluster, non-sharding techniques are often needed as well.

We study 3 well-known non-sharding techniques that reduce peak GPU memory: activation checkpointing, mixed precision training, and CPU offloading. Apart from implementing each technique from scratch to better understand its

mechanism and potential optimizations, we aim to answer the problem of choosing the combination of techniques to use in order to efficiently navigate the trade-off space between peak GPU memory usage and training efficiency.

While the 3 memory optimization techniques in our consideration are in use by many existing ML frameworks and systems, it remains challenging to find good technique combinations and configurations. As detailed in Section 2, there are existing works that automatically find good configurations within a single technique, and those that automatically plan across 2 techniques. However, to the best of our knowledge, no prior work has automated the process of technique-selecting among all 3 techniques.

To address the technique-picking question, we need to study the effect of each memory optimization technique and those of combining them. Therefore, we design and implement MemOpt to enable easy technique composition. Based on our system, we conduct benchmarking on different technique combinations and configurations to record peak memory usage and per-iteration training latency.

Our measurements show that mixed precision training alone provides modest peak memory savings while reducing training time, activation checkpointing saves slightly more memory but incurs a modest slowdown, and CPU offloading achieves the largest memory reduction at the cost of significant slowdown. Based on these results, we suggest a simple heuristic: enable mixed precision training first, add activation checkpointing if additional memory savings are needed, and only then enable CPU offloading if memory remains a bottleneck.

When all 3 techniques are used, MemOpt can train a 2.8B-parameter variant of our homework Transformer—2.31× larger than the largest trainable model when no memory optimizations are enabled.

2 Background

Memory Optimization Algorithms

Memory optimization algorithms for large-scale training have evolved to address the growing gap between model size and GPU memory capacity. Among all methods of reducing peak memory usage in non-parallel training fashion, there are three algorithms being most widely used: **CPU-offloading** moves optimizer states or activations from GPU to CPU memory to reduce GPU usage; **Mixed-precision training** stores and computes with lower-precision tensors to shrink memory footprint; **Activation checkpointing**

¹We do rely on the basic building blocks in PyTorch.

saves only a subset of activations and recomputes the rest during backward to cut memory consumption.

Intra-algorithm Automatic planning

Intra-algorithm automatic planning refers to systems that optimize a single memory-optimization algorithm by automatically determining how it should be applied to a particular model and hardware setup. Instead of relying on manually chosen checkpoints, precision settings, or tensor-placement rules, these systems generate execution plans that balance memory savings with overhead. For activation checkpointing, **Checkmate** computes near-optimal recomputation schedules via integer linear optimization [1]. **Dynamic Tensor Rematerialization (DTR)** adapts checkpointing online by tracking memory pressure and deciding which tensors to retain or evict [2]. For mixed-precision training, **AMPT-GA** explores precision configurations with a genetic search to maximize FP16/BF16 usage while preserving stability [3]. Together, these systems demonstrate how automatic intra-algorithm planning can outperform static configurations by tailoring each technique to model structure, memory behavior, and runtime conditions.

Inter-algorithm Automatic Selection

Inter-algorithm automatic selection refers to systems that choose combinations of memory-optimization algorithms rather than tuning a single method in isolation. These systems explore a broader design space, determining not only how an individual algorithm should be applied but which algorithms should be used together to best satisfy memory constraints with minimal performance loss. **AutoHete** is a representative example, automatically selecting a mix of activation checkpointing and CPU offloading for parameters and optimizer states to fit GPU memory budgets while limiting overhead [8]. **Capuchin** take this idea further by making tensor-level decisions about whether each tensor should be recomputed or offloaded to CPU, avoiding reliance on fixed checkpointing patterns [6]. By jointly reasoning over tradeoffs among recomputation, data movement, and memory capacity, inter-algorithm planners can outperform any single optimization applied in isolation. However, to the best of our knowledge, no system yet provides auto-planning across all 3 techniques which is studied in our work.

3 Design

3.1 Overview

Figure 1 shows the high-level workflow that applies different techniques to our homework Transformer model.

We introduce 3 APIs: `checkpoint`, `MixedPrecisionTrainer`, and `CPUOffloadingTrainer`. Our API design goals are to provide ease of use and composability. Thus we design each API as a wrapper around a PyTorch module.

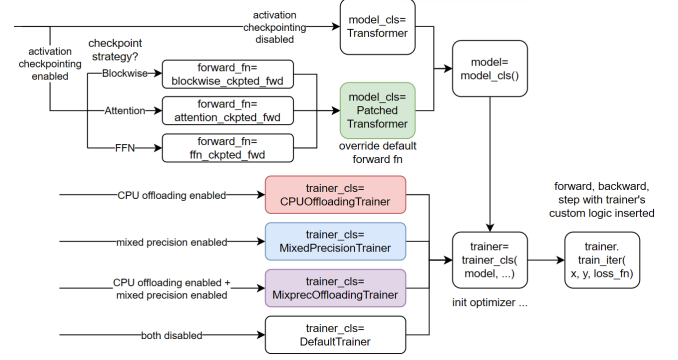


Figure 1. Overview of MemOpt: Workflow to compose different techniques.

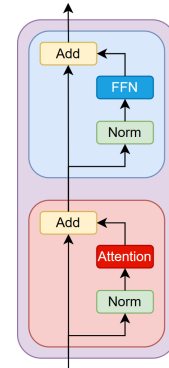


Figure 2. Checkpointing strategies: **Blockwise** checkpoints the entire Transformer block (purple), **Attention** checkpoints the attention sub-block (light red), **FFN** checkpoints the FFN sub-block (light blue).

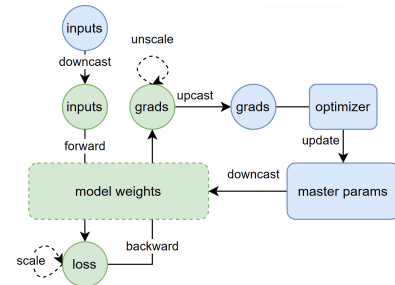


Figure 3. Mixed precision training: Blue tensors are in full precision, and green tensors are in lower precision. If auto-cast is used, only full precision weights are stored and lower precision weights are cast on the fly.

3.2 Activation Checkpointing Strategies

Our checkpoint API is a simplified version of the PyTorch `torch.utils.checkpoint.checkpoint` API. It disables gradient tracking within the checkpointed module during forward computation, thus the activations are not saved. During

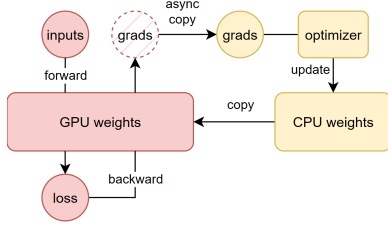


Figure 4. CPU offloading: Red tensors are on GPU, yellow tensors are on CPU. Gradients are immediately copied to CPU once produced.

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
    from memopt.actckpt import checkpoint
    x = self.token_embeddings(token_ids) # (batch_size, seq_len, d_model)
    for layer in self.layers:
        x = layer(x)
        x = checkpoint(layer, x)
    x = self.ln_final(x)
    logits = self.lm_head(x)
    return logits
```

Figure 5. Diff between the forward function of the Blockwise checkpointing strategy and the original Transformer.forward

the backward pass, it recomputes the forward pass with gradient tracking enabled to build the gradient graph for the checkpointed module, and then it launches backward on the graph.

As shown in Figure 2, we design 3 activation checkpointing strategies implemented with our checkpoint API. 1) **Blockwise**: For each Transformer block, only the input to the block is stored. 2) **Attention**: The attention modules in each block are checkpointed—only the input to the attention modules is stored; all activations from the FFN modules are stored. 3) **FFN**: The FFN modules in each block are checkpointed.

For each checkpointing strategy, we create a patched Transformer class that replaces the Transformer’s original forward function with a custom forward function containing the checkpointing logic. Figure 5 shows the patched forward function for the Blockwise strategy.

3.3 Mixed Precision Trainer

Figure 3 shows our mixed precision training flow. We design 2 variants of our MixedPrecisionTrainer. 1) **No-autocast**: Following the design in [5], we keep model weights in lower precision and maintain a copy of “master” parameters in FP32. For a training iteration, forward and backward passes are computed in lower precision, while the optimizer (with FP32 states) updates the master parameters in full-precision arithmetic. At the end of the backward pass, lower-precision gradients are upcast to FP32 and then attached to the master parameters. At the end of the optimizer update, the master parameters are downcast and copied into the lower-precision weights. 2) **Autocast**: We keep model weights in full-precision (thus no master copy) and use torch.autocast

API	Description
checkpoint(fn, *tensors)	Run fn(tensors) under checkpointing and return the result.
MixedPrecisionTrainer(model, dtype, autocast=True, loss_scale=None)	Mixed precision training wrapper. dtype can be FP16 or BF16.
CPUOffloadingTrainer(model)	CPU offloading wrapper.

Table 1. Key MemOpt APIs

to perform casting on-the-fly for all tensors.

To avoid gradient underflow, our MixedPrecisionTrainer provides the option of loss scaling. It also has the option to specify the lower precision dtype between FP16 and BF16.

3.4 CPU Offloading Trainer

As shown in Figure 4, the CPUOffloadingTrainer offloads gradients and optimizer states to the CPU. The CPU has a copy of the model parameters. After the forward and backward passes are computed on the GPU, each gradient is transferred asynchronously to CPU using backward hooks. The CPU updates its copy of parameters after all gradients arrive. The updated parameters are copied back to the GPU for the next iteration.

3.5 Technique Composition

For the case where mixed precision and CPU offloading are both enabled, we implement a MixprecOffloadTrainer following a similar design in [7]. This is similar to the no-autocast variant of MixedPrecisionTrainer; the difference is that the full-precision gradients, optimizer states, and master parameters (the blue tensors in Figure 3) are now on the CPU, and that offloading logic is added. To reduce the amount of memory copying between the CPU and GPU, MixprecOffloadTrainer always copies weights and gradients in lower precision instead of full precision. This is done by downcasting parameters before copying them back to the GPU, and upcasting the gradients after receiving them on the CPU.

Figure 1 shows the workflow of composing all 3 techniques together. If an activation checkpointing strategy is used, we patch the transformer’s forward function to add the corresponding checkpointing logic. Depending on whether each of mixed precision and CPU offloading is enabled, we select the trainer to use among CPUOffloadingTrainer, MixedPrecisionTrainer, MixprecOffloadingTrainer, and DefaultTrainer. Here, the DefaultTrainer executes the standard training loop after setting up the optimizer. Lastly, we wrap the model inside the trainer and are ready to train.

Size	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
large	1280	5120	36	20

Figure 6. Model sizes used in evaluation

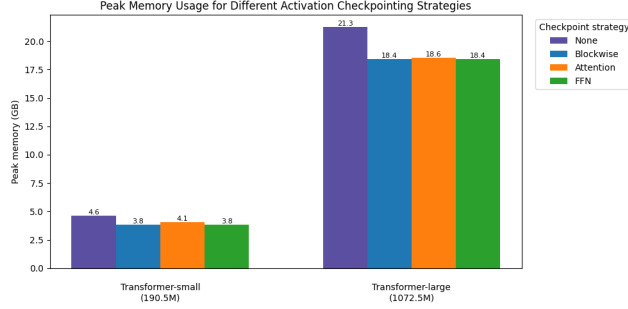


Figure 7. Peak memory usage with different activation checkpointing strategy

4 Implementation

Based on our homework infrastructure, we implement the memory optimization techniques from scratch using basic PyTorch building blocks. Some key APIs are listed in Table 1. Our system consists of the following key modules:

- `memopt.actckpt` implements activation checkpointing using a custom `torch.autograd.Function`. It disables gradient tracking in the forward pass and recomputes activations with gradient tracking enabled in the backward pass.
- `memopt.mixprec` implements mixed precision training as a wrapper around a model. This wrapper handles the logic for tensor casting, loss scaling, and overflow checking.
- `memopt.offload` implements CPU offloading of optimizer states and gradients. It handles asynchronous GPU-CPU memory copying.

Our code is available at <https://github.com/AndyUB/memopt>.

5 Evaluation

We focused on evaluating memory and latency performance of individual algorithm, and tradeoffs between memory and latency across combinations of memory saving algorithms. We tested on two sizes of Transformer model as shown in Figure 6 from homework, the small one contains 190 million parameters while the large one contains 1 billion parameters. We used random data for experiments. Our baseline is FP32 model trained with no memory optimizations.

5.1 checkpoint

Activation checkpointing consistently lowers peak memory consumption across all strategies, reducing memory usage

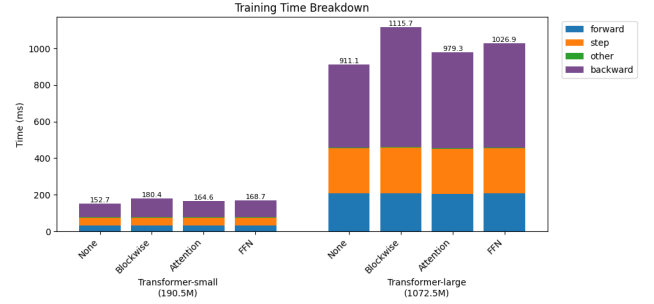


Figure 8. End-to-end latency with different activation checkpointing strategy

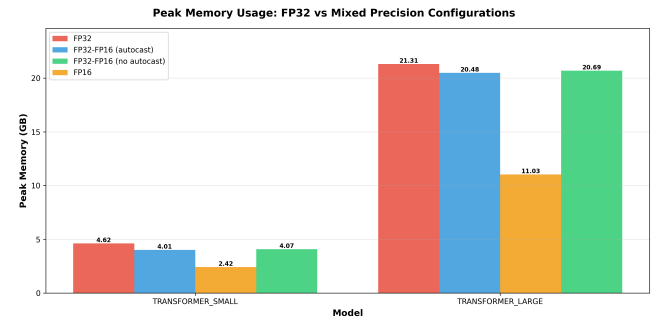


Figure 9. Peak memory usage with different mixed-precision configurations

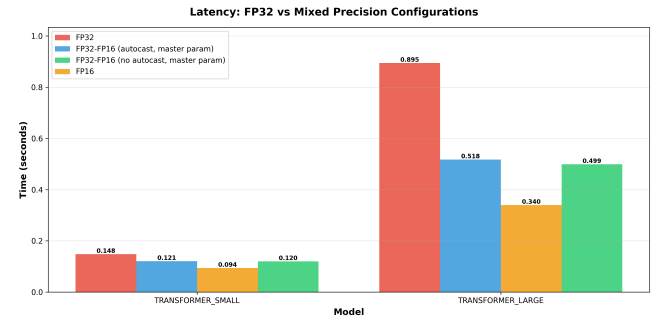


Figure 10. End-to-end latency with different mixed-precision configurations

by roughly 12–17 percent compared to no checkpointing as shown in Figure 7. Figure 8 shows that this comes with a corresponding increase in training time due to additional recomputation during backpropagation. Blockwise checkpointing introduces the largest latency overhead, while attention-only and FFN-only checkpointing add more moderate slowdowns. FFN checkpointing is strictly better than Blockwise since FFN has less recomputation overhead while they both offer the same peak memory reduction.

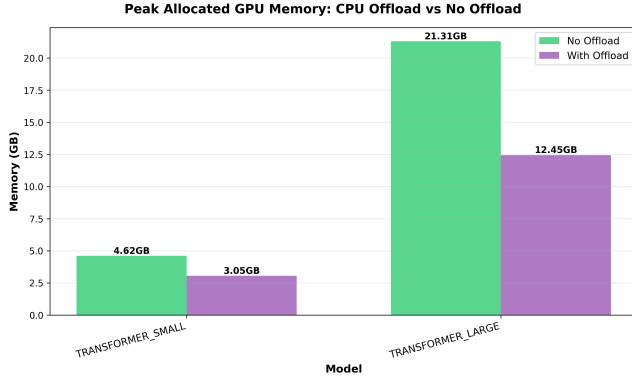


Figure 11. Peak memory usage with and without CPU offload

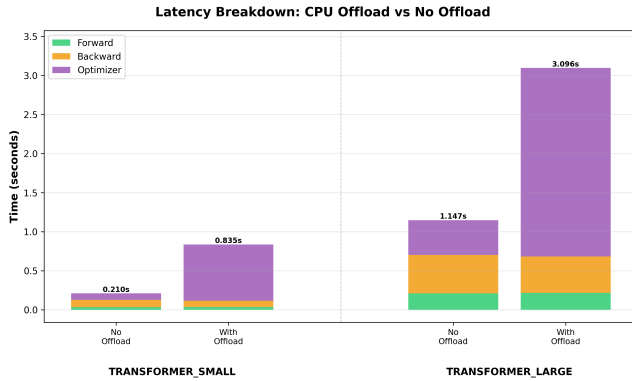


Figure 12. End-to-end latency with and without CPU offload

5.2 MixedPrecisionTrainer

Because the GPUs used in our experiments do not support BF16, our BF16 results are unexpected and we exclude them here. The FP16 results in Figure 9 and Figure 10 show that mixed precision both improves training efficiency and reduces peak memory usage across both Transformer models. The no-autocast variant has slightly higher peak memory (since it stores both lower precision weights and a copy of master params), but it is slightly faster than the autocast variant. Both variants reduce peak memory usage by 3%-13%. Note though FP16 is both the fastest and has the least peak memory, [5] argues that unlike FP32-FP16 approaches, it results in a non-trivial decrease in model accuracy.

5.3 CPUOffloadingTrainer

The CPU offloading results in Figure 11 show a substantial reduction in peak GPU memory usage for both model sizes, dropping by roughly 34 percent for the small model and over 40 percent for the large model. This demonstrates that moving optimizer states to CPU is effective for alleviating GPU memory pressure, especially at larger scales. However, this benefit comes with a significant latency increase. The stacked

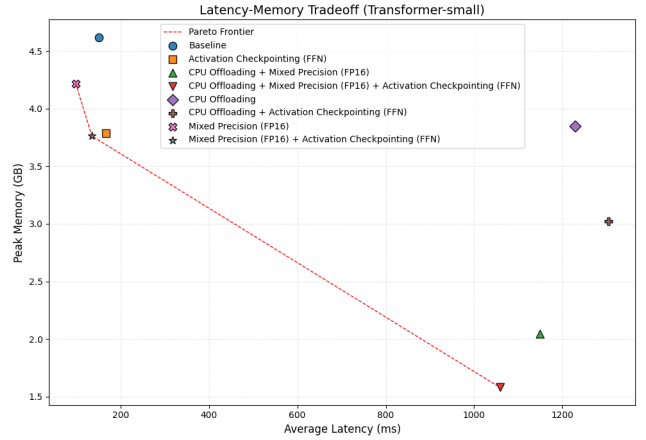


Figure 13. Pareto frontier for Transformer size small

timing results indicate that while forward and backward computation remain similar, the optimizer step grows dramatically under offloading because of repeated host-device transfers. As a result, end-to-end latency increases by more than three times for the small model and almost triples for the large model as shown in Figure 12.

5.4 Pareto Frontier

Figures 13 and 14 present the memory-latency Pareto frontiers across all technique combinations for training Transformer-small and Transformer-large, respectively. Notably, using mixed-precision alone achieves the lowest latency (even lower than the baseline); combining all 3 techniques together results in the largest peak memory reduction.

With the configuration for other techniques fixed, compared with the case of disabling mixed precision, enabling it achieves both lower latency and greater peak memory reduction. Therefore, our heuristic suggests always enabling mixed precision training (unless it leads to significantly worse model accuracy). A reminder here is to check whether BF16 is supported by the GPUs and if not, use FP16 instead.

We also observe that compared to the modest overhead of activation checkpointing, CPU offloading results in significant slowdown. Thus our heuristic prefers applying the activation checkpointing optimization before CPU offloading.

6 Discussion

6.1 Limitations

The current design evaluates each memory optimization technique in isolation or in simple combinations, but it does not explore their interactions at a finer level of granularity. CPU offloading is applied uniformly rather than selectively to different parameter groups or optimizer states, so the experiments do not capture adaptive offload schedules or approaches that overlap communication with computation. Activation checkpointing is limited to fixed patterns and

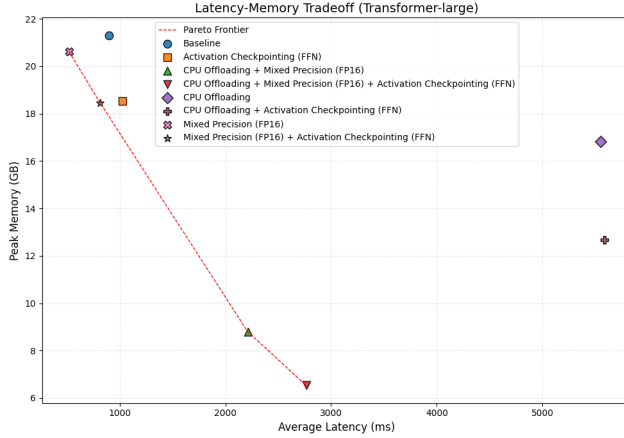


Figure 14. Pareto frontier for Transformer size large

does not account for layer-wise activation sizes or runtime variability. Mixed precision is treated as a global choice, without investigating layer-specific precision assignments or the numerical stability limits of each configuration. These simplifications reduce the generality of the measured tradeoffs and have distance to the complexity of real training systems.

6.2 Future Work

Future research can focus on automatic systems that jointly optimize mixed precision, checkpoint placement, and CPU offloading on more fine-grained level. Another direction is to implement offloading with more aggressive ZenFlow-style [4] compute and communication overlap that not only overlaps copying with backward, but also with next forward when combined with restricted delayed update. Extending the runtime to support dynamic decisions at the tensor level would enable checkpoint locations, precision levels, and offload choices to adapt to memory pressure in real time. Finally, evaluating these strategies on larger models and multi-GPU settings would reveal additional bottlenecks such as interconnect bandwidth and kernel scheduling behavior that are not visible in the present single-GPU study.

7 Conclusion

MemOpt provides a unified framework for composing activation checkpointing, mixed precision training, and CPU offloading to reduce peak GPU memory usage during Transformer training. By implementing each technique on top of basic PyTorch building blocks and enabling flexible combinations, the system reveals clear tradeoffs between memory savings and latency. Mixed precision offers the best efficiency gains, activation checkpointing produces additional but modest memory reductions with moderate slowdown, and CPU offloading achieves the largest savings at the cost of significant latency. Together, these results motivate a simple

practical heuristic: enable mixed precision first, add activation checkpointing when memory remains constrained, and apply CPU offloading only when further reductions are required. Using all three techniques, MemOpt successfully trains a model more than twice the size of the baseline limit.

References

- [1] Ankit Bhardwaj, Weiyang Wang, Jeremy Carin, Adam Belay, and Manya Ghobadi. 2025. Checkmate: Zero-Overhead Model Checkpointing via Network Gradient Replication. arXiv:2507.13522 [cs.DC] <https://arxiv.org/abs/2507.13522>
- [2] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2021. Dynamic Tensor Rematerialization. arXiv:2006.09616 [cs.LG] <https://arxiv.org/abs/2006.09616>
- [3] Pradeep V Kotipalli, Ranvijay Singh, Paul Wood, Ignacio Laguna, and Saurabh Bagchi. 2019. AMPT-GA: automatic mixed precision floating point tuning for GPU applications. In *Proceedings of the ACM International Conference on Supercomputing (Phoenix, Arizona) (ICS '19)*. Association for Computing Machinery, New York, NY, USA, 160–170. doi:10.1145/3330345.3330360
- [4] Tingfeng Lan, Yusen Wu, Bin Ma, Zhaoyuan Su, Rui Yang, Tekin Bicer, Masahiro Tanaka, Olatunji Ruwase, Dong Li, and Yue Cheng. 2025. ZenFlow: Enabling Stall-Free Offloading Training via Asynchronous Updates. arXiv:2505.12242 [cs.DC] <https://arxiv.org/abs/2505.12242>
- [5] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. arXiv:1710.03740 [cs.AI] <https://arxiv.org/abs/1710.03740>
- [6] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 891–905. doi:10.1145/3373376.3378505
- [7] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. arXiv:2101.06840 [cs.DC] <https://arxiv.org/abs/2101.06840>
- [8] Zihao Zeng, Chubo Liu, Xin He, Juan Hu, Yong Jiang, Fei Huang, Kenli Li, and Wei Yang Bryan Lim. 2025. AutoHete: An Automatic and Efficient Heterogeneous Training System for LLMs. arXiv:2503.01890 [cs.LG] <https://arxiv.org/abs/2503.01890>